Proximal Methods

1 Proximal Operator

Consider a composite minimization problem:

$$F(x) = f(x) + h(x) \to \min_x$$

where $f \in C^1$ and convex, $h \in C, h \notin C^1$, convex and "simple". The term "simple" will be explained later.

Suppose that $f \in C_L^{1,1}$ (has Lipschitz constant L). We can then consider a task for the upper bound:

$$f(x) + h(x) \leq m_k(x) = f(x_k) + \langle \nabla f(x_k), x - x_k \rangle + \frac{L}{2}\|x - x_k\|^2 + h(x) \to \min_x$$

The idea of proximal methods is that this task can sometimes be solved efficiently when we know the structure of $h(x)$. Let us simplify:

$$m_k(x) = \frac{L}{2}\left(\langle x, x\rangle - 2\langle x, x_k\rangle\right) + \langle \nabla f(x_k), x\rangle + h(x) + \text{const} =$$
$$= \frac{L}{2}\left(\langle x, x\rangle - 2\langle x, x_k - \frac{1}{L}\nabla f(x_k)\rangle\right) + h(x) + \text{const} \to \min_x$$

Dividing by L, and wrapping the norm, we get:

$$\frac{1}{2}\left\|x - \left(x_k - \frac{1}{L}\nabla f(x_k)\right)\right\|^2 + \frac{1}{L}h(x) \to \min_x$$

Let us define a **proximal operator**:

$$\text{prox}_h(x) := \arg\min_u \left(\frac{1}{2}\|u - x\|^2 + h(u)\right)$$

We suppose that the structure of $h(x)$ allows us to compute this operator efficiently. For example, it could be separated into a sum of independent one-dimensional minimization problems. The name "proximal" is used because we are searching for a value somewhere in the vicinity of x.

Our derivations above give rise to the **proximal gradient method**:

$$\boxed{x_{k+1} = \arg\min_x \; m_k(x) = \text{prox}_{\frac{h}{L}}\left(x_k - \frac{1}{L}\nabla f(x_k)\right)}$$

We want function $h(x)$ to be "simple", i.e. we can efficiently compute the proximal operator of h.

Proximal gradient method is a descent algorithm:

$$F(x_{k+1}) \leq m_k(x_{k+1}) \leq \{\text{minimization on k'th step}\} \leq m_k(x_k) = F(x_k)$$

Let us consider some examples of a proximal operator.

1) $h(x) = 0 \implies \text{prox}_h(x) = x$ (identity)

This implies that standard gradient descent is a special case of a proximal gradient method.

2)

$$h(u) = I_c(u) = \begin{cases} u \in C: & 0 \\ u \notin C: & +\infty \end{cases} \implies F(x) = f(x) + I_c(x) \to \min_x \iff f(x) \to \min_{x \in C}$$

Thus, the projected gradient method is also a special case of the proximal gradient method.

3)

$$h(u) = \lambda \|u\|_1 \implies \text{prox}_h(x) = \arg\min_u \left(\frac{1}{2} \|u - x\|_2^2 + \lambda \|u\|_1 \right) = \{ \arg\min_{u_i} \left[\frac{1}{2} (x_i - u_i)^2 + \lambda \|u_i\| \right] \}$$

So, the problem separates into one-dimensional problems. Let us consider 1D task:

$$f(u) = \frac{1}{2}(u - x)^2 + \lambda |u| \to \min_u$$

The criterion of the minimum is that zero vector is a subgradient: $0 \in \partial f(u)$.

$$0 \in \partial f(u) = u - x + \lambda \begin{cases} u > 0: & 1 \\ u < 0: & -1 \\ u = 0: & [-1, 1] \end{cases}$$

This can be expressed as:

$$u_* = \begin{cases} x > \lambda: & x - \lambda \\ x < -\lambda: & x + \lambda \\ |x| \leq \lambda: & 0 \end{cases}$$

The proximal operator is visualized in Fig. 1. It is visible where the shrinkage comes from, when L1 regularization is used.

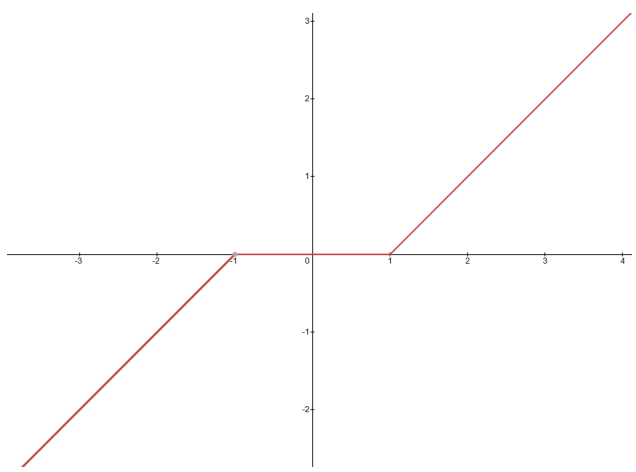

Figure 1: Visualization of a proximal operator for the L1 norm ($\lambda = 1$)

2 Convergence of the Proximal Gradient Method

Consider the update of the proximal gradient method with some learning rate α, which we have to choose as we typically don't know the Lipschitz constant L:

$$y = \text{prox}_{\alpha h}(x - \alpha \nabla f(x))$$

and define $G_\alpha(x)$ in the following way:

$$y = \text{prox}_{\alpha h}(x - \alpha \nabla f(x)) = x - \alpha G_\alpha(x)$$

where $G_\alpha(x) := \frac{1}{\alpha}(x - y)$ is called the **gradient mapping**.

Let us now write the optimality condition for the local task, which minimizes y (zero vector is a subgradient):

$$y = \arg\min_y \left(\frac{1}{2}\|y - \alpha\nabla f(x)\|^2 + \alpha h(y) \right) \iff 0 \in y - (x - \alpha\nabla f(x)) + \alpha\partial h(y)$$

Divide everything by α and substitute $G_\alpha(x)$:

$$G_\alpha(x) - \nabla f(x) \in \partial h(y) \iff G_\alpha(x) \in \nabla f(x) + \partial h(y) = \partial F(x) \tag{1}$$

We can see that substituting $G_\alpha(x) = 0$ implies the minimum of the total function $F(x)$. Thus, we have proved one implication of the following statement:

Statement 1.

$$G_\alpha(x) = 0 \iff x = \arg\min_x F(x)$$

The reverse implication will be proved later.

We can think of $G_\alpha(x)$ as some analogue of the ordinary gradient in the sense that we want it to approach zero. We can also use it for the stopping criterion: $\|G_\alpha(x)\|_2^2 \le \epsilon^2$.

Let us now derive the convergence rate of the proximal gradient method.

Suppose $f \in C_L^{1,1}$ and μ-strongly convex, and $F(y) = f(y) + h(y)$. We also assume that learning rate α is sufficiently small:

$$\alpha \le \frac{1}{L}$$

Consider the next step:

$$y = x - \alpha G_\alpha(x)$$

Lower bound:

$$F(y) = f(y) + h(y) \le \{f \in C_L^{1,1}\} \le f(x) + \langle \nabla f(x), y - x\rangle + \frac{L}{2}\|y - x\|^2 + h(y) =$$
$$= \{y = x - \alpha G_\alpha(x)\} = f(x) - \langle \nabla f(x), \alpha G_\alpha(x)\rangle + \frac{L\alpha^2}{2}\|G_\alpha(x)\|^2 + h(y) \le \{\alpha \le \frac{1}{L}\} \le$$
$$\le f(x) - \alpha\langle \nabla f(x), G_\alpha(x)\rangle + \frac{\alpha}{2}\|G_\alpha(x)\|^2 + h(y)$$

Due to strong convexity, we have:

$$f(z) \ge f(x) + \langle \nabla f(x), z - x\rangle + \frac{\mu}{2}\|z - x\|^2 \implies f(x) \le f(z) - \langle \nabla f(x), z - x\rangle - \frac{\mu}{2}\|z - x\|^2 \tag{2}$$

Also, by the definition of the subdifferential, we know:

$$h(z) \ge h(y) + \langle \partial h(y), z - y\rangle = \{\text{Eq. 1: } G_\alpha(x) - \nabla f(x) \in \partial h(y)\} = h(y) + \langle G_\alpha(x) - \nabla f(x), z - y\rangle$$

Where we used that the equation holds for every subgradient, so we substituted a subgradient based on G_α that we have derived in Eq. 1. From this, we get:

$$h(y) \le h(z) - \langle G_\alpha(x) - \nabla f(x), z - y\rangle \tag{3}$$

Now we substitute Eq. 2 and Eq. 3 into the bound for $F(y)$:

$$F(y) \le f(z) - \langle \nabla f(x), z - x\rangle - \frac{\mu}{2}\|z - x\|^2 - \alpha\langle \nabla f(x), G_\alpha(x)\rangle + \frac{\alpha}{2}\|G_\alpha(x)\|^2 + h(z) - \langle G_\alpha(x) - \nabla f(x), z - y\rangle \tag{4}$$

The following equation holds for all x and for all z. Substituting $z = x$ gives:

$$F(y) \le f(x) + h(x) - \alpha\langle \nabla f(x), G_\alpha(x)\rangle + \frac{\alpha}{2}\|G_\alpha(x)\|^2 - \langle G_\alpha(x), x - y\rangle + \langle \nabla f(x), x - y\rangle$$

Using that $\alpha G_\alpha(x) = x - y$, the terms with $\nabla f(x)$ disappear, and we get a simplified expression:

$$\boxed{F(y) \leq F(x) - \frac{\alpha}{2}\|G_\alpha(x)\|^2} \tag{5}$$

Where y is the next step after applying the proximal operator. This equation shows us how much proximal gradient method improves at each step.

In particular, substituting $x = x_*$ (optimal value) and using the optimality gives us:

$$F(x_*) \leq F(y) \leq F(x_*) - \frac{\alpha}{2}\|G_\alpha(x)\|^2 \implies \|G_\alpha(x_*)\|^2 = 0$$

This gives us the reverse implication in the statement.

Substituting $z = x_*$ into Eq. 4, we get:

$$F(y) \leq F_* - \langle \nabla f(x), x_* - x\rangle - \frac{\mu}{2}\|x_* - x\|^2 - \alpha\langle \nabla f(x), G_\alpha(x)\rangle + \frac{\alpha}{2}\|G_\alpha(x)\|^2 - \langle G_\alpha(x), x_* - y\rangle + \langle \nabla f(x), x_* - y\rangle$$

In the right side, we use $y = x - \alpha G_\alpha(x)$ to simplify a part of it:

$$-\alpha\langle \nabla f(x), G_\alpha(x)\rangle + \langle \nabla f(x), x_* - y\rangle = \langle \nabla f(x), x_* - x + \alpha G_\alpha(x) - \alpha G_\alpha(x)\rangle = \langle \nabla f(x), x_* - x\rangle$$

Note that this term consides with the second term from the right side. Therefore, it simplies:

$$F(y) \leq F_* - \frac{\mu}{2}\|x_* - x\|^2 + \frac{\alpha}{2}\|G_\alpha(x)\|^2 - \langle G_\alpha(x), x_* - y\rangle$$

Now, substitute $G_\alpha(x) = \frac{y-x}{\alpha}$:

$$F(y) \leq F_* - \frac{\mu}{2}\|x_* - x\|^2 + \frac{1}{2\alpha}\|x - y\|^2 - \frac{1}{\alpha}\langle x - y, x_* - y\rangle$$

Let us now complete the square for the final 2 terms on the right side:

$$\begin{aligned}
&\frac{1}{2\alpha}\|x - y\|^2 - \frac{1}{\alpha}\langle x - y, x_* - y\rangle = \\
&= \frac{1}{2\alpha}\left[\|x - y\|^2 - 2\langle x - y, x_* - y\rangle + \|x_* - y\|^2 - \|x_* - y\|^2\right] = \\
&= \frac{1}{2\alpha}\left[\|x - x_*\|^2 - \|x_* - y\|^2\right]
\end{aligned}$$

Grouping the variables, we get:

$$F(y) \leq F_* + \frac{1}{2\alpha}\left[(1 - \alpha\mu)\|x - x_*\|^2 - \|y - x_*\|^2\right]$$

By definition of the optimum, we know that $F_* \leq F(y)$. That implies:

$$0 \leq F(y) - F_* \leq \frac{1}{2\alpha}\left[(1 - \alpha\mu)\|x - x_*\|^2 - \|y - x_*\|^2\right]$$

Equivalently:

$$\|y - x_*\|^2 \leq (1 - \alpha\mu)\|x - x_*\|^2$$

So, we have a linear convergence rate. Furthermore, if we put the maximum possible (and the best possible) learning rate $\alpha = 1/L$, we get the speed $1 - \frac{\mu}{L}$ — exactly like in standard gradient descent for strongly-convex functions.

However, in reality we do not know the Lipschitz constant L. Therefore, we need to somehow adapt the learning rate. One way is called L-adaptation procedure:

Algorithm 1 L-adaptation procedure (for each step)

Initialize L
while True **do**

$$y_k = \text{prox}_{\frac{1}{L_k}h}\left(x_k - \frac{1}{L_k}\nabla f(x_k)\right)$$

if

$$f(y_k) \leq f(x_k) + \langle \nabla f(x_k),\, y_k - x_k\rangle + \frac{L_k}{2}\|y_k - x_k\|^2$$

then
break
else
$L_k \leftarrow \gamma L_k$
end if
end while
$L_{k+1} \leftarrow L_k/\gamma$

3 Proximal Newton Method

Suppose we want to apply the Newton method to non-smooth function $F(x) = f(x) + h(x)$. We can approximate:

$$F(x) = f(x) + h(x) \approx m_k(x) = f(x_k) + \langle \nabla f(x_k), x - x_k\rangle + \frac{1}{2}\langle H_k(x - x_k), x - x_k\rangle + h(x) \to \min_x$$

This leads to the minimization of the **scaled proximal operator** $\text{prox}_h^H(x)$:

$$\text{prox}_h^H(x) := \arg\min_u \left(\frac{1}{2}\langle H(u - x), u - x\rangle + h(u)\right)$$

And the update typically has form:

$$\hat{x}_{k+1} = \arg\min_x m_k(x) = \text{prox}_L^H\left(x_k - H_k^{-1}\nabla f(x_k)\right)$$
$$x_{k+1} = x_k + \alpha_k(\hat{x}_{k+1} - x_k)$$

Where α_k is the analogue of a learning rate.

We use the same stopping criterion:

$$G_\alpha(x) = \frac{1}{\alpha}\left(x - \text{prox}_{\alpha h}(x - \alpha\nabla f(x)\right) \qquad \|G_\alpha(x)\|^2 \leq \epsilon^2 \implies \text{stop}$$

However, computing the scaled proximal operator is often impossible analytically. Therefore, we can iteratively solve the local task:

$$m_k(x) = \langle \nabla f(x_k), x - x_k\rangle + \frac{1}{2}\langle H_k(x - x_k), x - x_k\rangle + h(x) \to \min_x$$

We can view this as a proximal task:

$$m_k(x) = S(x) + h(x) \to \min_x$$

So, we can use the proximal gradient method:

$$x_{k+1} = \text{prox}_{\frac{h}{L}}\left(x_k - \frac{1}{L}\nabla s(x_K)\right)$$
$$\nabla s(x) = \nabla f(x_k) + H_k(x - x_k) \quad \text{(we only need hess-vec product)}$$

and monitor the gradient mapping $\hat{G}_{1/L}(x_k)$:

$$\hat{G}_{1/L}(x_k) = L\left(x_k - \text{prox}_{h/L}\left[x_k - \frac{1}{L}\nabla s(x_k)\right]\right)$$

with the stopping criterion relative to the global task mapping:

$$\|\hat{G}_{1/L}(x)\| \leq \eta_k \|G_{1/L}(x_k)\|$$

for some scalar η_k.

4 ADMM

Consider the following optimization task:

$$f(x) + g(z) \to \min_{x,z}$$
$$s.t.\ \ Ax + Bz = C$$

where f and g are convex non-smooth functions. We suppose that we can solve each of the optimization tasks on its own easily (only minimize f, or only minimize g), but we need to somehow "merge" the solutions with the linear constraint.

One example is LASSO, which can be formulated as:

$$\frac{1}{2N}\|Xw - y\|_2^2 + \lambda\|z\|_1 \to \min_{w,z}$$
$$s.t.\ \ w = z$$

We define the **augmented Lagrangian function** $L_\rho(x, z, \mu)$:

$$L_\rho(x, z, \mu) := f(x) + g(z) + \langle \mu, Ax + Bz - C\rangle + \frac{\rho}{2}\|Ax + Bz - C\|_2^2$$

Note that the addition of ρ term makes this function strongly-convex, and therefore there is a unique minimum by (x, z), with fixed Lagrange multiplier μ. Let us denote it with $d_\rho(\mu)$:

$$d_\rho(\mu) = \inf_{x,z} L_\rho(x, z, \mu)$$

It has the following properties:

1. $d_\rho(\mu) \geq d(\mu) = \inf_{x,z} L(x, z, \mu)$
2. $d_\rho(\mu) \leq f(x_*) + g(z_*)$
3. $\nabla_\mu d_\rho(\mu) = Ax_*(\mu) + Bz_*(\mu) - C$

First property can be obtained from:

$$L_\rho(x, z, \mu) = L(x, z, \mu) + \frac{\rho}{2}\|Ax + Bz - C\| \geq L(x, z, \mu)$$

and then taking infimum for both sides.

The second property is obtained from the minimality in $d_\rho(\mu)$: if we substitute (x_*, z_*), which obviously are feasible, and therefore satisfy the linear constraint:

$$d_\rho(\mu) \leq L_\rho(x_*, z_*, \mu) = f(x_*) + g(x_*) + \langle \mu, 0\rangle + \frac{\rho}{2}\|0\|^2 = f(x_*) + g(x_*)$$

The third property is evident from the linearity on μ.

ADMM step has the following form:

$$
\begin{aligned}
x_{k+1} &= \arg\min_x L_\rho(x, z_k, \mu_k) \\
z_{k+1} &= \arg\min_z L_\rho(x_{k+1}, z, \mu_k) \\
\mu_{k+1} &= \mu_k + \rho\left(Ax_{k+1} + Bz_{k+1} - C\right)
\end{aligned}
$$

Note that the step on μ has the form:

$$\mu_{k+1} = \mu_k + \rho \nabla_\mu d_\rho(\mu_k)$$

which is the gradient ascent with step size ρ. The reason why exactly ρ learning rate is used will be discussed later.

For the initial task, we have the following Lagrangian:

$$L(x, z, \mu) = f(x) + g(z) + \langle \mu, Ax + Bz - C \rangle$$

KKT conditions of optimality give us (assume differentiable f and g for simplicity):

$$
\begin{cases}
\nabla_x L(x, z, \mu) = \nabla f(x) + A^T \mu = 0 \\
\nabla_z L(x, z, \mu) = \nabla g(z) + B^T \mu = 0 \\
Ax + Bz = C
\end{cases}
$$

From the second ADMM step on z, obtaining z_{k+1}:

$$z_{k+1} = \arg\min_z L_\rho(x_{k+1}, z, \mu_k)$$

using optimality condition, we get:

$$
\begin{aligned}
0 &= \nabla_z L_\rho(x_{k+1}, z_{k+1}, \mu_k) = \nabla g(z_{k+1}) + B^T \mu_k + \rho B^T \left(Ax_{k+1} + Bz_{k+1} - C\right) = \\
&= \nabla g(z_{k+1}) + B^T \left[\mu_k + \rho\left(Ax_{k+1} + Bz_{k+1} - C\right)\right]
\end{aligned}
$$

Now, we see the reason why ρ step size is used. Then, the condition becomes:

$$\nabla g(z_{k+1}) + B^T \mu_{k+1} = 0$$

which means that (z_{k+1}, μ_{k+1}) always satisfy the optimality condition.

Now, let us see what happens with x_{k+1}. From optimality condition, we have:

$$
\begin{aligned}
x_{k+1} = \arg\min_x L_\rho(x, z_k, \mu_k) \implies 0 &= \nabla_x L_\rho(x_{k+1}, z_k, \mu_k) = \\
&= \nabla f(x_{k+1}) + \rho A^T \left(Ax_{k+1} + Bz_k - C\right) = \nabla f(x_{k+1}) + A^T \left[\mu_k + \rho\left(Ax_{k+1} + Bz_k - C\right)\right]
\end{aligned}
$$

Unfortunately, here we got z_k instead of z_{k+1}, so we can't just substitute μ_{k+1}. We need to add a residual:

$$0 = \nabla f(x_{k+1}) + A^T \mu_{k+1} + \rho A^T B(z_k - z_{k+1})$$

The term $\rho A^T B(z_k - z_{k+1})$ tells us how "close" the variables are to satisfying KKT conditions, and can be considered as a residual.

We define the following two residuals:

$$
\begin{aligned}
r_{k+1} &:= Ax_{k+1} + Bz_{k+1} - C && \textbf{(primal residual)} \\
s_{k+1} &:= \rho A^T B(z_k - z_{k+1}) && \textbf{(dual residual)}
\end{aligned}
$$

They both show how far the variables are from being optimal: primal residual shows error in the linear constraint, while dual residual shows error in optimality by x.

The stopping criterion for ADMM can the be used as the following:

$$\|r_{k+1}\| \leq \epsilon,\ \|s_{k+1}\| \leq \epsilon \implies \text{stop}$$

The hyperparameter ρ shows how much we enforce the linear constraint. If ρ is small, the augmented Lagrangian is close to the ordinary Lagrangian, and the linear constraint is weakly enforced (Lagrange multiplier μ is slowly updated). On the contrary, if ρ is large, the constraint is strongly enforced and the method is forced from the start to look at close to $Ax + Bz = C$ values.

It is possible to use constant ρ and treat it as a hyperparameter. However, a more sophisticated approach is to use ρ **adaptation procedure** every M-th iteration:

$$\rho_{new} = \begin{cases} \|s_{k+1}\| \leq \gamma \|r_{k+1}\| & : \rho_{old} \times \tau \\ \|r_{k+1}\| \leq \gamma \|s_{k+1}\| & : \rho_{old}/\tau \end{cases}$$

Here we adapt the ρ value dynamically based on the behavior of the primal and dual residuals. If one is much greater than the other, we change ρ accordingly to focus more on minimizing that larger residual.

Typical values are $\gamma = 10, \tau = 2$ and M somewhere from 10 to 50.

Once we have updated ρ, we also need to update μ as it was obtained using additions of $\rho \times (\ldots)$:

$$\rho \to \rho \times \tau \implies \mu \to \mu/\tau \qquad \rho \to \rho/\tau \implies \mu \to \mu \times \tau$$

5 ADMM Examples

5.1 ADMM for LASSO

Consider the LASSO task (in ADMM form):

$$\begin{aligned} &\frac{1}{2N}\|Xw - y\|_2^2 + \lambda\|z\|_1 \to \min_{w,z} \\ &s.t.\ \ w = z \end{aligned}$$

Write the augmented Lagrangian:

$$L_\rho(w, z, \mu) = \frac{1}{2N}\|Xw - y\|^2 + \lambda\|z\|_1 + \langle \mu, w - z\rangle + \frac{\rho}{2}\|w - z\|^2$$

The update step on Lagrange multiplier μ has form:

$$\mu_{k+1} = \mu_k + \rho(w_{k+1} - z_{k+1})$$

For clarity, let us define the rescaled multiplier as $u_k := \mu_k/\rho$. Then, the update will have form:

$$u_{k+1} = u_k + w_{k+1} - z_{k+1}$$

The augmented Lagrangian will become:

$$\begin{aligned} L_\rho(w, z, \mu) &= \frac{1}{2N}\|Xw - y\|^2 + \lambda\|z\|_1 + \rho\langle u, w - z\rangle + \frac{\rho}{2}\|w - z\|^2 \\ &= \frac{1}{2N}\|Xw - y\|^2 + \lambda\|z\|_1 + \frac{\rho}{2}\left[\|w - z\|^2 + 2\langle u, w - z\rangle + \|u\|^2 - \|u\|^2\right] \\ &= \frac{1}{2N}\|Xw - y\|^2 + \lambda\|z\|_1 + \frac{\rho}{2}\|w - z + u\|^2 - \frac{\rho}{2}\|u\|^2 \end{aligned}$$

Note that the last term with $\|u\|^2$ does not participate in optimization by x and z and can be ignored.

Now let us derive the steps. First, minimize by w:

$$w_{k+1} = \arg\min_w L_\rho(w, z_k, u_k)$$

This implies

$$\nabla_w L_\rho = \frac{1}{N} X^T (Xw - y) + \rho(w - z + u) = 0$$

From here we derive the analytical solution by x:

$$w_{k+1} = \left(\frac{1}{N} X^T X + \rho I\right)^{-1} \left(\frac{1}{N} X^T y + \rho(z_k - u_k)\right)$$

Note that here ρ also takes part as regularization for the covariance matrix. If ρ is constant and the computational resources allow us, we can do Cholesky decomposition of the regularized covariance matrix once, and then perform this step very quickly.

For adaptive ρ, it Cholesky wouldn't work, so we need to perform eigendecomposition of the covariance matrix:

$$\frac{1}{N} X^T X = Q^T \Lambda Q$$

Then, we can easily compute it for regularized case:

$$\frac{1}{N} X^T X + \rho I = Q^T (\Lambda + \rho I) Q$$

As for step on z, here we have the following task:

$$z_{k+1} = \arg\min_z L_\rho(w_{k+1}, z, u_k)$$

Where the minimization is equivalent to:

$$\lambda \|z\|_1 + \frac{\rho}{2} \|w_{k+1} + u_k - z\|^2 \to \min_z$$

Notice that this is a proximal operator for L_1 norm. Therefore, the closed form for z_{k+1} is:

$$z_{k+1} = \text{prox}_{\lambda/\rho \|\cdot\|_1}(w_{k+1} + u_k)$$

And for u, as was already mentioned, the update is:

$$u_{k+1} = u_k + w_{k+1} - z_{k+1}$$

So, we have a full closed form for all of the ADMM iterations.

5.2 ADMM for parallel NN training

Consider the parallel NN training with an objectives $f_i(w)$ for $i = 1, \ldots, N$ and a shared regularizer $R(w)$:

$$\sum_{i=1}^N f_i(w) + R(w) \to \min_w \iff \begin{cases} \sum_{i=1}^N f_i(w_i) + R(z) \to \min_{w_1, \ldots, w_k, z} \\ w_i = z, \quad i = 1, \ldots, N \end{cases}$$

We again use the reparameterization $u = \mu/\rho$ to obtain:

$$L_\rho(w, z, u) = \sum_{i=1}^N f_i(w_i) + R(z) + \frac{\rho}{2} \|z - (w_i + u_i)\|^2$$

Minimization on each of w_i gives:

$$w_{k+1,i} = \arg\min_{w_i} f_i(w_i) + \frac{\rho}{2} \|w_i - z_{k+1} + u_{k+1,i}\|^2$$

which can be performed in parallel on different machines after obtaining z and u_i.

The most interesting part is combining the solutions to z on the master machine. The objective is:

$$R(z) + \frac{\rho}{2}\sum_{i=1}^{N}\|z - (w_i + u_i)\|^2 \to \min_z$$

Expanding the norm, we can rewrite it to:

$$R(z) + \frac{\rho}{2}\sum_i [\langle z, z\rangle - 2\langle z, w_i + u_i\rangle] + \text{const} = R(z) + \frac{\rho N}{2}\left[\langle z, z\rangle - 2\langle z, \frac{1}{N}\sum_{i=1}^{N}(w_i + u_i)\rangle\right] + \text{const} =$$

$$= R(z) + \frac{\rho N}{2}\|z - \frac{1}{N}\sum_{i=1}^{N}(w_i + u_i)\|^2 + \text{const} \to \min_z$$

We can easily see that this is a proximal operator for R, so the solution is:

$$z_{k+1} = \text{prox}_{R/(\rho N)}\left(\frac{1}{N}\sum_{i=1}^{N}(w_i + u_i)\right)$$

And for u, as in previous task, the update is:

$$u_{k+1,i} = u_{k,i} + w_{k+1,i} - z_{k+1}$$

5.3 ADMM for NN quantization

Consider the case where we want to train a neural network with objective $f(w)$, such that all the weights are quantized. For example, the belong to

$$Q = \{-1, 0, 1\}$$

as in 1.57 bit format.

Using the indicator function ($+\infty$ when z not in Q, and 0 otherwise), the objective is:

$$f(w) + I_Q(z) \to \min_{w,z}$$
$$s.t.\ \ w = z$$

The augmented Lagrangian is:

$$L_\rho(w, z, u) = f(w) + I_Q(z) + \frac{\rho}{2}\|w - z + u\|^2$$

Optimization by w is a problem with quadratic penalty:

$$f(w) + \frac{\rho}{2}\|w - z + u\|^2 \to \min_w$$

And optimization by z is:

$$I_Q(z) + \frac{\rho}{2}\|z - (w + u)\|^2 \to \min_z$$

This is just a projection operator. We can write:

$$z = \text{proj}_Q(w + u)$$

And by u the update is as previously:

$$u_{k+1} = u_k + w_{k+1} - z_{k+1}$$

This can be more stable than straight-through estimator (STE) which is commonly used for quantization-aware training.